

实验报告

实验 8 流水线 CPU 设计与测试

课程名称

数字逻辑与计算机组成

指导教师

吴海军

1 实验目的

1. 掌握基础流水线 CPU 的设计方法。
2. 掌握基础流水线 CPU 中阻塞检测和旁路转发的实现方法。
3. 掌握基础流水线 CPU 分支静态预测实现方法。
4. 掌握基础流水线 CPU 程序代码优化的编程方法。
5. 掌握基础流水线 CPU 硬件设计优化的设计方法。

2 实验环境

平台：Logisim / RARS。

3 实验内容

3.1 流水线 cpu 设计实验

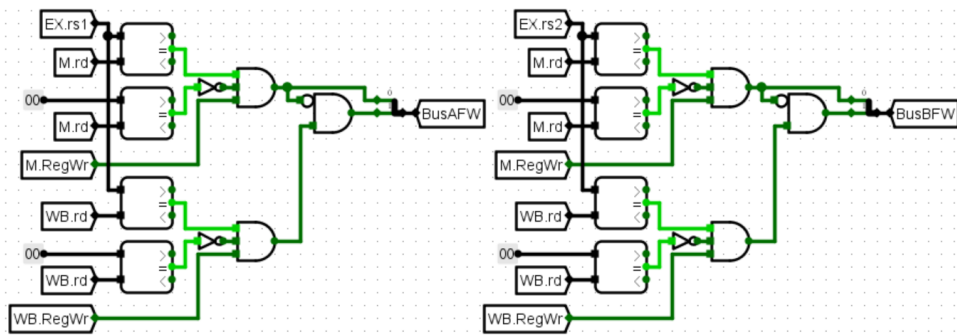
3.1.1 实验整体方案设计

- 设计转发、阻塞、跳转检测模块
- 设计流水段寄存器
- 设计流水线 cpu 整体数据通路

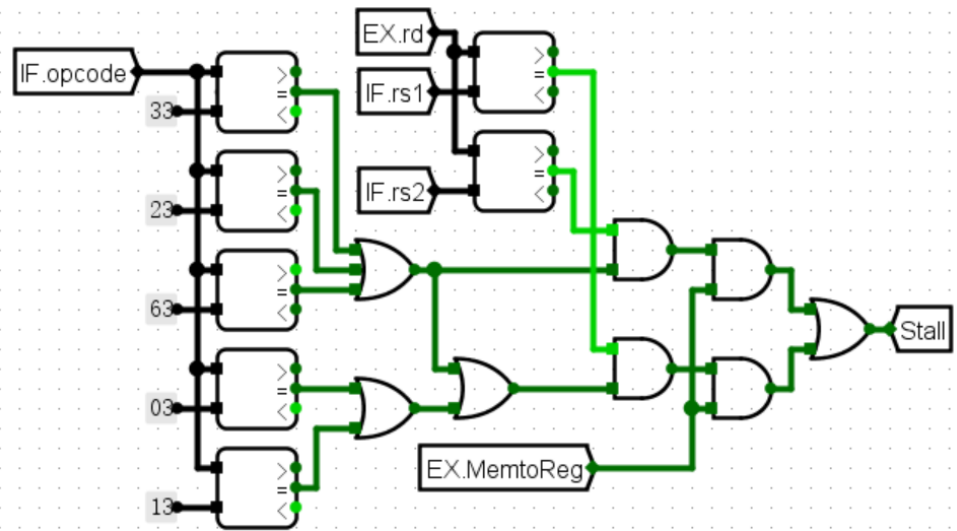
阶段	主要功能	关键信号
IF	取指、PC 更新	PC、Instr、IF/ID.PC+4、IF/ID.Instr
ID	译码、读寄存器、立即数生成	rs1、rs2、BusA、BusB、imm、ID/EX 控制信号
EX	ALU 运算、转发检测	ALU 结果、Zero、PC 目标地址、EX/M 控制信号
M	数据存储器访问、分支判断	Mem 地址、Mem 数据、MemtoReg、M/WB 控制信号
WB	结果写回寄存器堆	RegWr、rd、WB.Result、寄存器堆写信号

3.1.2 实验原理图和电路图

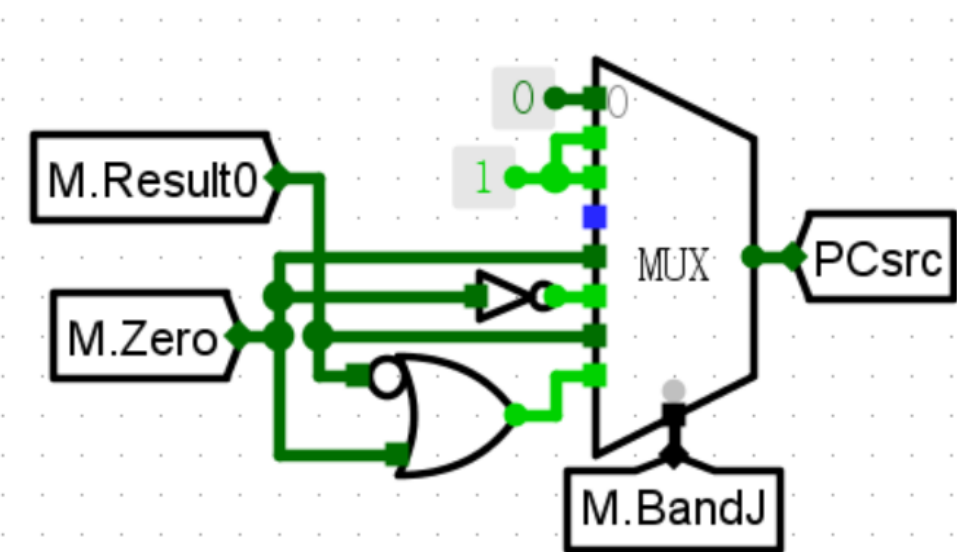
转发检测模块电路图



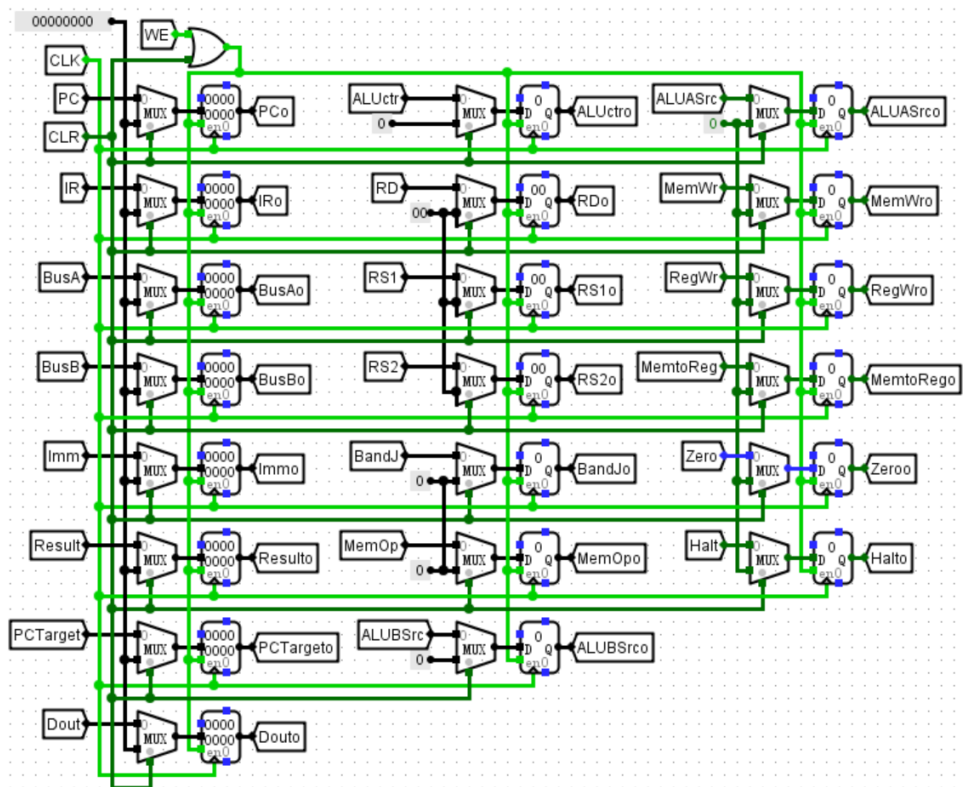
阻塞检测模块电路图



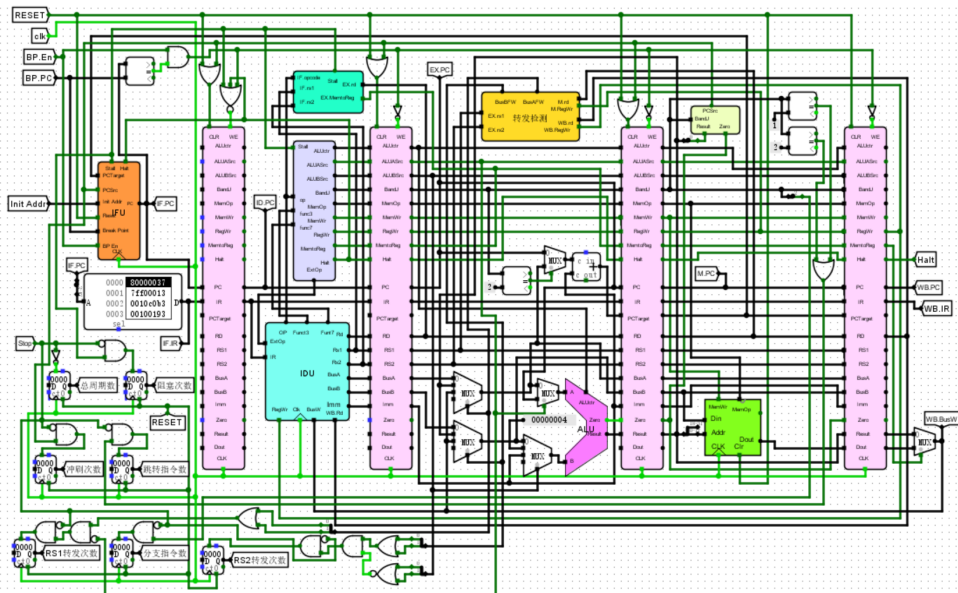
跳转检测模块电路图



流水线寄存器电路图



流水线 CPU 整体数据通路电路图



3.1.4 错误现象及分析

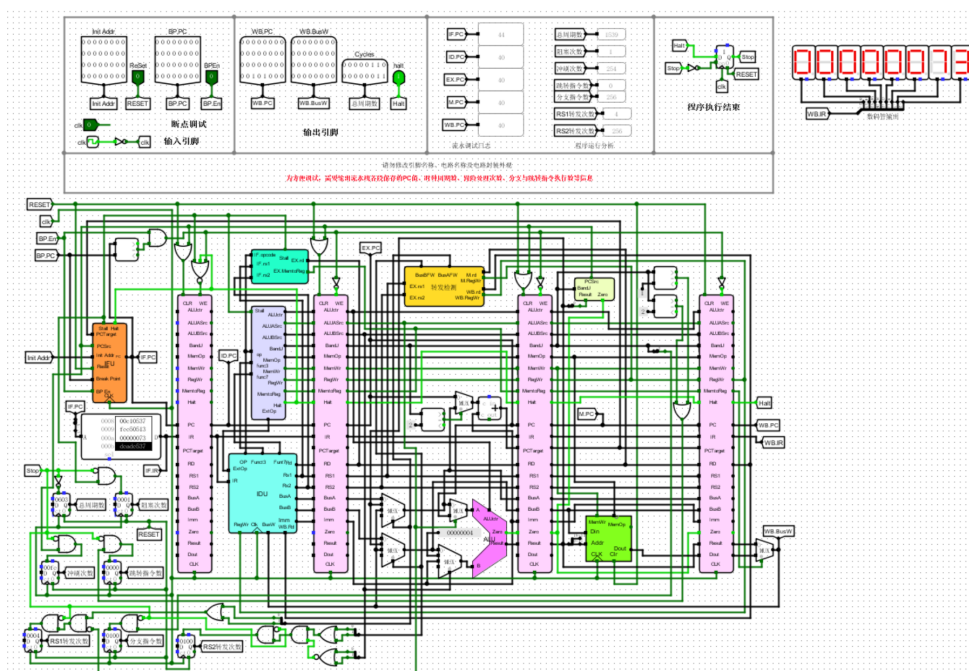
在 EX 段操作数被转发时，一开始的设计会导致使用立即数的指令可能错误地使用了转发的值，导致计算结果错误。分析原因是转发逻辑没有区分立即数指令和寄存器指令。

解决方法有两种，第一种是修改转发逻辑，使其在立即数指令时不进行转发，第二种是修改立即数指令的译码逻辑，先处理转发信号再处理 ALUBsrc 信号。最后选取的解决方案是第二种，这要求在对转发次数进行计数的时候要结合 ALUBsrc 信号一同判断是否成功进行了转发。

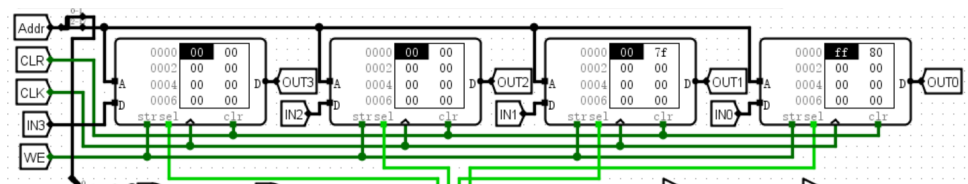
3.2 测试程序优化验证实验

3.2.1 实验数据仿真测试图

测试程序优化验证实验仿真测试图 1



测试程序优化验证实验仿真测试图 2



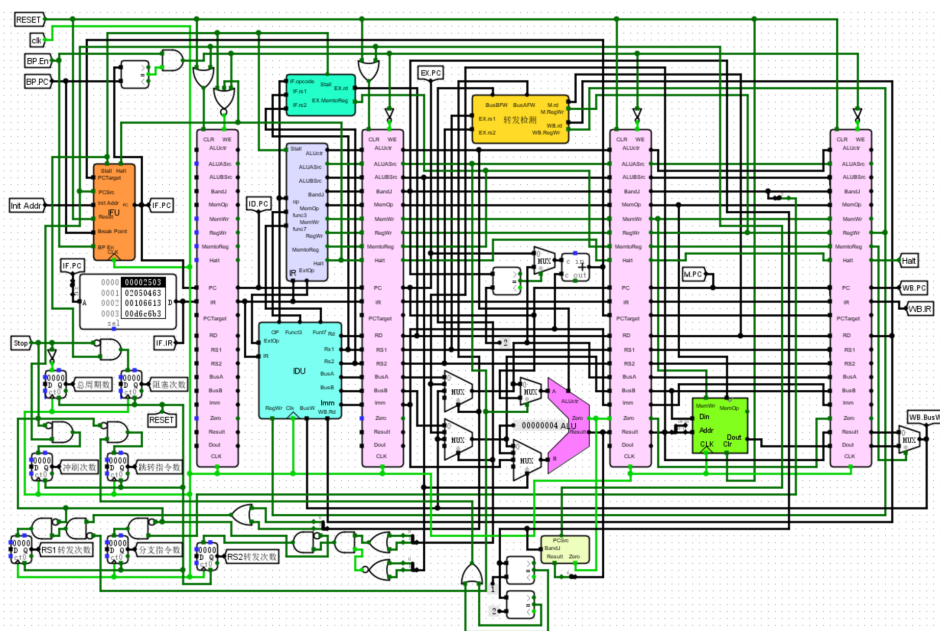
3.3 结构优化设计实验

3.3.1 实验整体方案设计

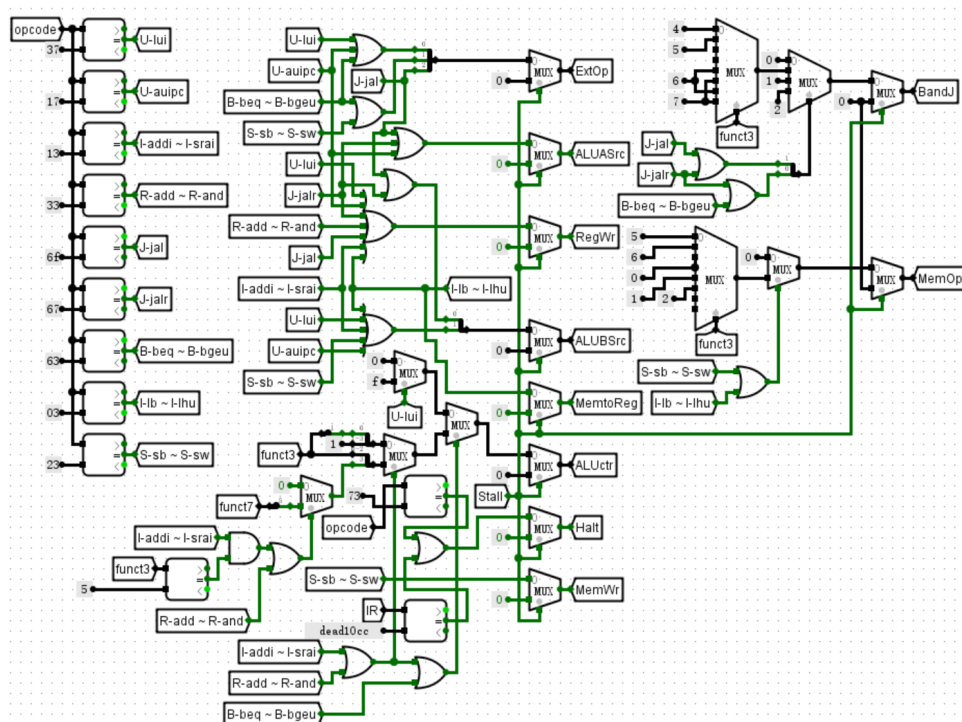
将 8.1 设计的流水线 CPU 的转发判断逻辑从 M 段迁移到 EX 段，减少转发判断的延迟，提升流水线 CPU 的运行频率。同时修改控制器电路，传入 IR，在 $IR = 0xdead10cc$ 时触发 halt 信号，停止流水线 CPU 的运行。

3.3.2 实验原理图和电路图

结构优化数据通路电路图

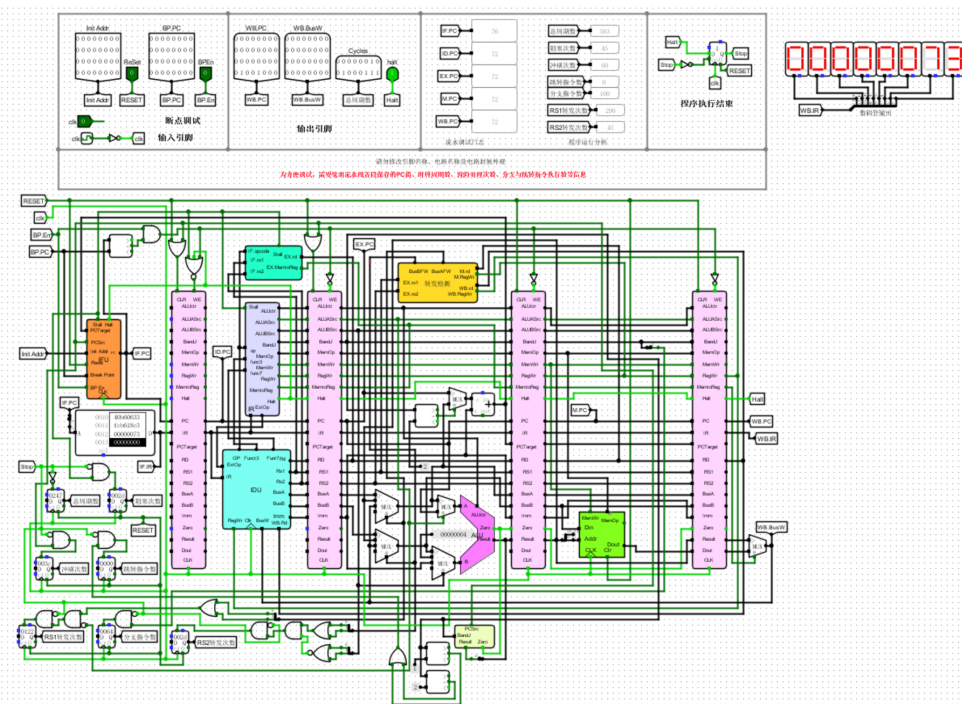


结构优化控制器电路图



3.3.3 实验数据仿真测试图

结构优化仿真测试图



3.4 官方测试验证实验

3.4.1 指令测试表格填写

序号	测试指令	x1 寄存器	x3 寄存器	时钟周期总数	阻塞次数	冲刷次数
1	add	0x00000010	0x00000026	493	0	16
2	addi	0x00000021	0x00000019	252	0	7
3	and	0x11111111	0x0000001b	513	0	16
4	andi	0x00ff00ff	0x0000000e	208	0	7
5	auipc	0x00000000	0x00000003	61	0	3
6	beq	0x00000003	0x00000015	341	0	27
7	bge	0x00000003	0x00000018	377	0	36
8	bgeu	0x00000003	0x00000018	402	0	36
9	blt	0x00000003	0x00000015	341	0	27
10	bltu	0x00000003	0x00000015	366	0	27
11	bne	0x00000003	0x00000015	345	0	29
12	jal	0x00000003	0x00000003	57	0	3
13	jalr	0x00000000	0x00000007	137	0	13
14	lb	0x00002000	0x00000013	257	2	7
15	lbu	0x00002000	0x00000013	257	2	7
16	lh	0x00002000	0x00000013	269	2	7
17	lhu	0x00002000	0x00000013	276	2	7
18	lui	0xffff800	0x00000006	63	0	1
19	lw	0x00002000	0x00000013	279	2	7
20	or	0x11111111	0x0000001b	516	0	16
21	ori	0x00ff00ff	0x0000000e	215	0	7
22	sb	0x00000001	0x00000017	452	0	13
23	sh	0x00003001	0x00000017	505	0	13
24	sll	0x00000400	0x0000002b	521	0	16
25	slli	0x00000021	0x00000019	251	0	7
26	slt	0x00000010	0x00000026	487	0	16
27	slti	0x00ff00ff	0x00000019	247	0	7
28	sltiu	0x00ff00ff	0x00000019	247	0	7
29	sltu	0x00000010	0x00000026	487	0	16
30	sra	0x00000400	0x0000002b	540	0	16
31	srai	0x00000021	0x00000019	266	0	7
32	srl	0x00000400	0x0000002b	534	0	16
33	srli	0x00000021	0x00000019	260	0	7
34	sub	0x00000010	0x00000025	485	0	16
35	sw	0x12233001	0x00000017	512	0	13
36	xor	0x11111111	0x0000001b	515	0	16
37	xori	0x00ff00ff	0x0000000e	217	0	7

4 思考题

4.1 思考题 1

转发检测模块中，没有根据操作码 OP 来判断当前指令是否操作 rs1 和 rs2 字段，可能对输出产生什么影响？会导致程序执行错误吗？如何解决该问题，是否必要？被阻塞的指令能产生有效转发信号吗？

影响：store 指令 rs2 是数据，branch 指令 rs1/rs2 是比较操作数，不判断 OP 会导致错误转发信号，EX 阶段使用错误操作数，程序出错。解决：转发检测中增加 OP 判断，R 型/I 型指令操作 rd，store 操作 rs2，branch 操作 rs1/rs2，jal/jalr 操作 rd。必要性：必要，错误转发会覆盖数据通路值。阻塞转发：Stall 时阻塞指令停留 IF/ID 段，转发信号未生成，不产生错误转发。

4.2 思考题 2

本实验中将跳转检测放在 EX 段执行，能否继续优化迁移到 ID 段执行？试分析可行性。

可行性：理论上可将跳转检测迁移到 ID 段以减少分支延迟损失。ID 段需具备：读取 rs1/rs2 值（需旁路）、计算分支目标地址（需加法器）、比较操作数（需比较器）。障碍：ID 段无独立 ALU，需添加比较器和加法器增加硬件复杂度；且 ID 段时序紧张，同时进行译码/读寄存器/立即数生成，再加入分支判断可能拖慢时钟周期。结论：技术可行但需额外硬件支持，将分支检测放在 EX 段是合理折中。

4.3 思考题 3

根据基础流水线 CPU 中累加和测试程序三种情形的执行性能，分别讨论总周期数、阻塞次数、rs1 转发次数、rs2 转发次数、分支和跳转指令执行次数数据准确性，计算程序的平均 CPI，并讨论提升该测试程序执行效率的方法。

三种情形： $n=0$ 、 $n=1$ 、 $n \geq 2$ ，循环次数不同导致性能数据差异。准确性：总周期数准确；阻塞次数由 Load-Use 冒险决定，准确可数；转发次数准确反映 RAW 冒险频率；分支/跳转执行次数在每次循环迭代中均执行，累计准确。CPI：总周期数/指令总数，开销主要来自 Load-Use 阻塞。优化：load 结果直接前递减少阻塞；优化循环结构减少控制冒险；编译器调度独立指令填补间隙；BTB 减少分支损失。

4.4 思考题 4

如何修改流水线 CPU 的设计，实现非法指令和结果溢出两种异常处理，标识异常类型，跳转异常处理程序。

非法指令：ID 段添加指令合法性检测，检查 opcode/funct3/funct7 组合，检测到时 Exception=1, cause=1, PC 保存至 mepc, Flush 流水线，跳转到 mtvec 入口。结果溢出：ALU 中增加溢出检测电路（有符号加法：操作数符号相同但结果符号不同即为溢出），Overflow=1 时 cause=2, 同样 Flush 并跳转。标识：CSR 的 cause 字段标识异常类型（1= 非法指令，2= 溢出），mepc 保存触发异常的 PC，mtval 保存辅助信息。

4.5 思考题 5

在实验中对于分支指令采用静态预测的方法来解决控制冒险，讨论如何实现 1 位动态预测的方法。

静态预测：实验中总是预测分支不成立，错误时冲刷流水线造成 2 周期损失。1 位动态预测：BHT 每项 1 位历史位，用 PC 低位索引，历史位 =1 预测跳转，=0 预测不跳转，分支结果出来后更新历史位，预测错误时 Flush 并取正确路径；BTB 记录目标地址减少计算延迟。优势：利用分支行为重复性，对循环分支比静态预测更准确；首次执行需冷启动，可结合 2 位饱和计数器提升准确性。

4.6 思考题 6

简述高级流水线设计方法。

1. 超标量：每周期发射多条指令提升 ILP，需解决发射顺序、寄存器重命名、转移预测问题。
2. VLIW：多条指令打包为长指令字，编译时显式指定并行，简化硬件但代码密度低。
3. 乱序执行：允许不按序执行避免数据冒险，通过寄存器重命名消除 WAW/WAR 冒险，需 ROB 和寄存器映射。
4. 深流水线：增加流水线级数提升频率，但增加冒险和分支损失，需权衡频率与单指令延迟。
5. 多线程：SMT 在同一核内并发多线程，共享执行单元弥补利用率不足，线程切换掩盖长延迟。
6. 高级转移预测：2 位饱和计数器、BHT、BTB、感知器预测器等，提升预测准确率减少控制冒险损失。